

**DDoS ATTACK DETECTION USING MACHINE
LEARNING:
A LEAKAGE-FREE MULTI-VIEW DEEP LEARNING
FRAMEWORK
(FLOWFUSION)**

A Project Report

DDoS Attack Detection using Machine Learning: A Leakage-Free
Multi-View Deep Learning Framework (FlowFusion)

A Project Report

Submitted in partial fulfilment of the requirements of the course AIPC 304,
B.Tech (Artificial Intelligence and Machine Learning), 6th Semester

By

Tanish, Ronit Yadav

123108004,123108025

Under the supervision of

Dr. Yogita

**Department of Computer Engineering
National Institute of Technology Kurukshetra**

July 2026

Certificate

This is to certify that the project report entitled “DDoS Attack Detection using Machine Learning: A Leakage-Free Multi-View Deep Learning Framework (FlowFusion)” is a bonafide record of the work carried out by Tanish, Roll No. 123108004, under the supervision of Dr. Yogita, Department of Computer Engineering, National Institute of Technology Kurukshetra, during the period from January 2026 to May 2026.

This work is original and has not been submitted elsewhere, in full or in part, for the award of any degree or diploma.

Signature of Candidates

Tanish (123108004)

Ronit Yadav (123108025)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: _____

Signature of the Faculty Mentor:

Dr. Yogita, Assistant Professor

Department of Computer

Engineering

NIT Kurukshetra

Table of Contents

Acknowledgement	5
Abstract	5
List of Figures	5
List of Tables	6
1. Introduction.....	7
2. Motivation.....	7
3. Related/Background Work	8
4. Proposed Work with Relevant Algorithms	8
4.1 Dataset.....	8
4.2 Leakage-Aware Preprocessing.....	9
4.3 FlowFusion: Proposed Multi-View Architecture	10
4.4 Baseline Models	12
4.5 Ensemble: FlowFusion + LightGBM.....	13
4.6 Explainability	13
5. Experimental Details	13
5.1 Setup and Evaluation Protocol	13
5.2 Why a Cross-Session Split.....	13
5.3 Main Results	14
5.4 Branch Ablation Study.....	14
5.5 Per-Class Results.....	15
5.6 Ensemble Results	15
5.7 Interpretability Analysis.....	19
5.8 Discussion: Why Gradient-Boosted Trees Remain Competitive	21
6. Conclusion	22
References.....	23

Acknowledgement

I would like to express my sincere gratitude to Dr. Yogita, Department of Computer Engineering, National Institute of Technology Kurukshetra, for the guidance, encouragement, and constructive feedback that shaped this project at every stage. I am also thankful to the faculty of the Department of Computer Engineering for providing the resources and environment necessary to carry out this work. Finally, I would like to thank my family and friends for their constant support throughout the course of this project.

Abstract

Distributed Denial of Service (DDoS) attacks, particularly reflection/amplification (DrDoS) variants, are a major threat to network availability. In our prior work on CICDDoS2019, when using a single capture session let duplicate flow leaks across train/test split, which gave near perfect accuracy score . In this work ,we have used January 12, 2019 session for training and March 11, 2019 session for testing , for better generalization and testing. We used four baseline models – XGBoost, LightGBM, a residual MLP, and a Feature Tokenizer Transformer. We benchmarked these baselines against our multi-view architecture, FlowFusion. FlowFusion fuses a tabular self-attention view, a feature-interaction view, and a structural feature-group view via cross-branch attention and interpretable scalar gating. Both gradient-boosted baselines and FT-Transformer outperformed FlowFusion on Macro-F1, LightGBM is best overall(0.9891) at a lower inference cost of any deep model. However, a soft-voting ensemble of FlowFusion and LightGBM(alpha =0.41:059) which reached to 0.9895. We observed in the analysis of gate-value and branch ablation , all three FlowFusion views are contributing with balanced gate weights(0.63/0.68/0.60), rather than having one dominant branch. These results indicate gradient boosted tress are highly competitive with deep architectures on structured, tabular data while FlowFusion adds interpretability and ensemble gain.

Keywords — DDoS detection; CIC-DDoS2019; multi-view learning; FlowFusion; gated fusion; gradient boosting; cross-session generalization; interpretability.

List of Figures

Figure 1. Distributions of high variance CIC-DDoS2019 flow features

Figure 2. Pairwise correlation matrix of high-variance flow features

Figure 3. FlowFusion architecture and data-flow diagram

Figure 4. Per-sample CPU inference latency across all five models

Figure 5. Per-class F1 score across all six models

Figure 6. One-vs-rest ROC curves for FlowFusion

Figure 7. Confusion matrix for FlowFusion

Figure 8. One-vs-rest ROC curves for LightGBM

Figure 9. Confusion matrix for LightGBM

Figure 10. ROC curves for the FlowFusion + LightGBM ensemble

Figure 11. Confusion matrix for the FlowFusion + LightGBM ensemble

Figure 12. Mean learned gate values for FlowFusion's three branches

Figure 13. Global TreeSHAP feature importance for LightGBM

Figure 14. TreeSHAP summary plot for the BENIGN class

List of Tables

Table 1. Model comparison on the held-out test session

Table 2. Branch ablation study validating the FlowFusion architecture

Table 3. FlowFusion per-class F1 on the test session

Table 4. Ensemble performance versus its two constituent models

1. Introduction

A DDoS attack floods a target's compute, memory, or bandwidth resources with massive traffic from many sources. In DDoS, reflection/amplification (DrDoS) type are damaging, in this type an attacker spoofs the victim's address and sends small requests to open services(LDAP, MSSQL, NetBIOS, generic UDP) that reply with large responses. Small group of attackers can generate enough traffic to completely disrupt a very high speed connection. Traditional DDoS detection methods that rely on signatures or fixed thresholds often fail because modern attacks imitate normal traffic or use new amplification techniques. Therefore, machine learning models are employed analyze the flow-level features such as packet counts, byte counts, etc., which can be computed quickly enough to support near real time detection.

This report focuses on four key aspects. First, it adopts a strict cross-session evaluation protocol on CICDDoS2019 dataset by training the models on traffic of January 12, 2019, and evaluate them on traffic captured on March 11, 2019. This setup reduces the risk of data leakage that can occur in randomly shuffled train test split and provides a more realistic assessment of model generalization. Second, the study presents **FlowFusion**, a multi-view deep learning framework that partitions a 70 dimensional network flow feature vector into three complementary feature groups, then processed through dedicated branches, and combine their representatives using cross-branch attention and scalar gating mechanisms. Third, the proposed framework is evaluated alongside two gradient-boosted tree models(XGBoost and LightGBM) and two deep learning baselines(FlowMLP and FT-Transformer). Finally, the report analyzes the experimental results to examine the relative performance of gradient-boosted tree models and deep learning architectures, highlighting the strengths and limitations of each approach for DDoS detection on the CICDDoS2019 dataset.

2. Motivation

Several limitations identified in existing studies and our previous work motivate the present work. First, many investigations using the CIC-DDoS2019 dataset use randomly shuffled train-test splits within a single capture session. Such evaluation protocols risk temporal leakage — highly similar flows end up in both the training and testing sets. This can inflate the reported accuracy beyond what the model would actually achieve on new traffic from a different time period. Second, most deep learning approaches for DDoS detection process all flow features through a single encoder. Not many studies have tried splitting the features into multiple groups and learning separate representations from each group. Third, most deep learning models for DDoS detection report high accuracy but do not explain which features drove each prediction. In a security setting, this lack of transparency can make it harder to trust the model's decisions.

Based on these observations, we designed our work around three choices: (1) a cross-session evaluation protocol to avoid leakage, (2) a multi-view feature learning framework that splits the flow features into different groups, and (3) attention and gating mechanisms that show how much each feature group contributes to the final prediction. The goal is to check whether these design choices actually help with detection performance and make the model easier to interpret.

3. Related/Background Work

Gradient-boosted trees like XGBoost and LightGBM are commonly used for network flow classification. XGBoost uses regularized gradient boosting with column sampling, while LightGBM speeds up training using histogram-based splits and leaf-wise tree growth. Both models work well with mixed feature types and provide built-in feature importance rankings, which is why we chose them as baselines..

Several recent studies apply deep learning directly to DDoS detection and inform the design choices made here. Kumar et al. [1] compare Vanilla LSTM, Stacked LSTM, and a DNN against classical baselines (Random Forest, AdaBoost, Gaussian Naive Bayes) on CICDDoS2019, motivated by LSTM's suitability for sequential traffic data; their LSTM model reaches approximately 98% accuracy. Unlike supervised approaches, Aktar and Nur [2] proposed a semi-supervised Deep Contractive Autoencoder trained solely on benign network traffic. The model identifies malicious flows by comparing reconstruction errors against a probabilistic threshold instead of relying on labelled attack samples during training. When evaluated on the CICDDoS2019 dataset, it achieved attack-wise accuracies ranging from 93.41% to 97.58%, outperforming conventional Autoencoder, Variational Autoencoder (VAE), and LSTM-based autoencoder models. Meenakshi et al. [3] present an empirical comparison, not a survey: CNN versus four recurrent variants (LSTM, BiLSTM, Stacked LSTM, GRU) on the Portmap.csv subset of CICDDoS2019, where Stacked-LSTM performs best. Building on these studies, our work evaluates an FT-Transformer baseline, a focal-loss objective, and a multi-branch gated-fusion framework on the complete six-class subset of CICDDoS2019. Unlike the studies above, we use a cross-session evaluation protocol rather than relying on a single attack category or randomly pooled session data.

The CICDDoS2019 dataset [4] contains network traffic from several DDoS attack types collected over two separate capture sessions. We worked directly with the original raw CSV files rather than using any externally preprocessed version, and applied the preprocessing steps described in Section 4.2. A common issue with this dataset is the choice of train-test split. If both the training and testing data come from the same capture session, temporally similar flows can end up in both sets, making the results look better than they actually are. We avoided this by using separate capture sessions for training and testing (Section 5.2). We call FlowFusion a multi-view framework rather than multimodal because all three branches receive the same 70-dimensional input. Each branch just learns a different representation of the same data.

4. Proposed Work with Relevant Algorithms

4.1 Dataset

This study utilizes the original CICDDoS2019 dataset [4], developed by the Canadian Institute for Cybersecurity. The dataset includes a diverse collection of DDoS attacks, such as reflection, amplification, and exploitation attacks, along with flow-level statistical features extracted from raw network traffic. These features include packet and byte counts, packet length statistics, inter-arrival times, TCP flag counts, and initial TCP window sizes. We conducted experiments using the original raw CICDDoS2019 dataset, which is processed after the leakage aware preprocessing pipeline described in Section 4.2 instead of relying on

an externally preprocessed version. To evaluate model generalization under realistic conditions, traffic captured on **January 12, 2019** is used exclusively for training, while data from **March 11, 2019** is reserved for testing. This cross-session split prevents overlap between the two sets and provides a more realistic assessment than evaluation within a single capture session.

The study focuses on the six traffic classes that contain sufficient samples in both sessions: **BENIGN**, **DrDoS_LDAP**, **DrDoS_MSSQL**, **DrDoS_NetBIOS**, **DrDoS_UDP**, and **Syn**. For training, a balanced subset of 25,000 flows is selected from each class, resulting in a total of 150,000 samples. In contrast, the test set retains its original class distribution to better reflect real-world network traffic and provide a realistic evaluation of model performance.

4.2 Leakage-Aware Preprocessing

In preprocessing, we removed identity/metadata columns that could let a model memorize hosts or sessions rather than learn traffic patterns. These columns include flow ID, source/destination IP, source port, timestamp, and the SimilarHTTP and Inbound fields. We converted infinite values to NaN and then replaced them using the median value computed from the data. To prevent data leakage, we applied all preprocessing parameters, including the median and scaling values, from the training dataset. As a result, no information from the test set influences the training process. After preprocessing, total 70 features are retained and used as input to the different models.

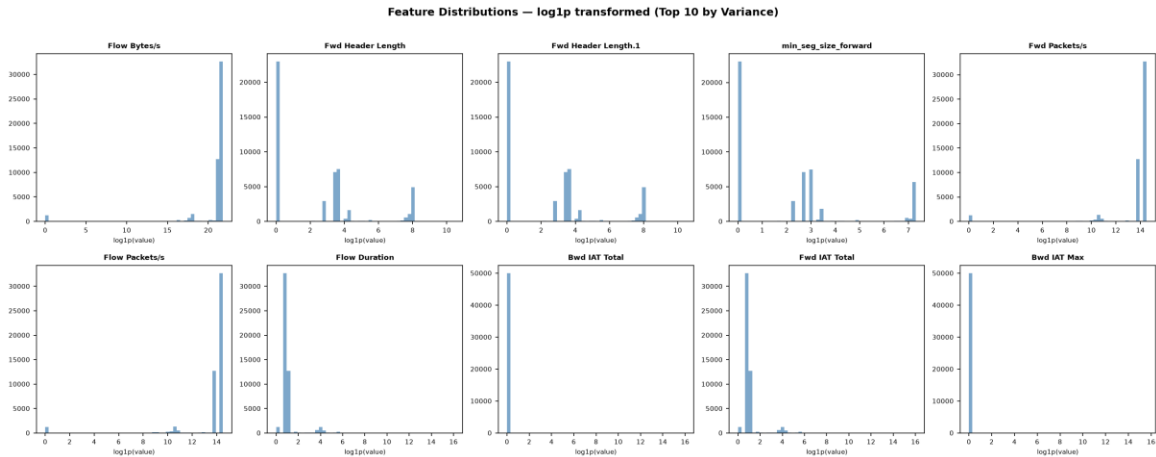


Figure 1. Log1p-transformed distributions of the ten highest-variance CIC-DDoS2019 flow features, illustrating the heavy-tailed, multi-modal statistics that motivate RobustScaler over standard z-score normalization.

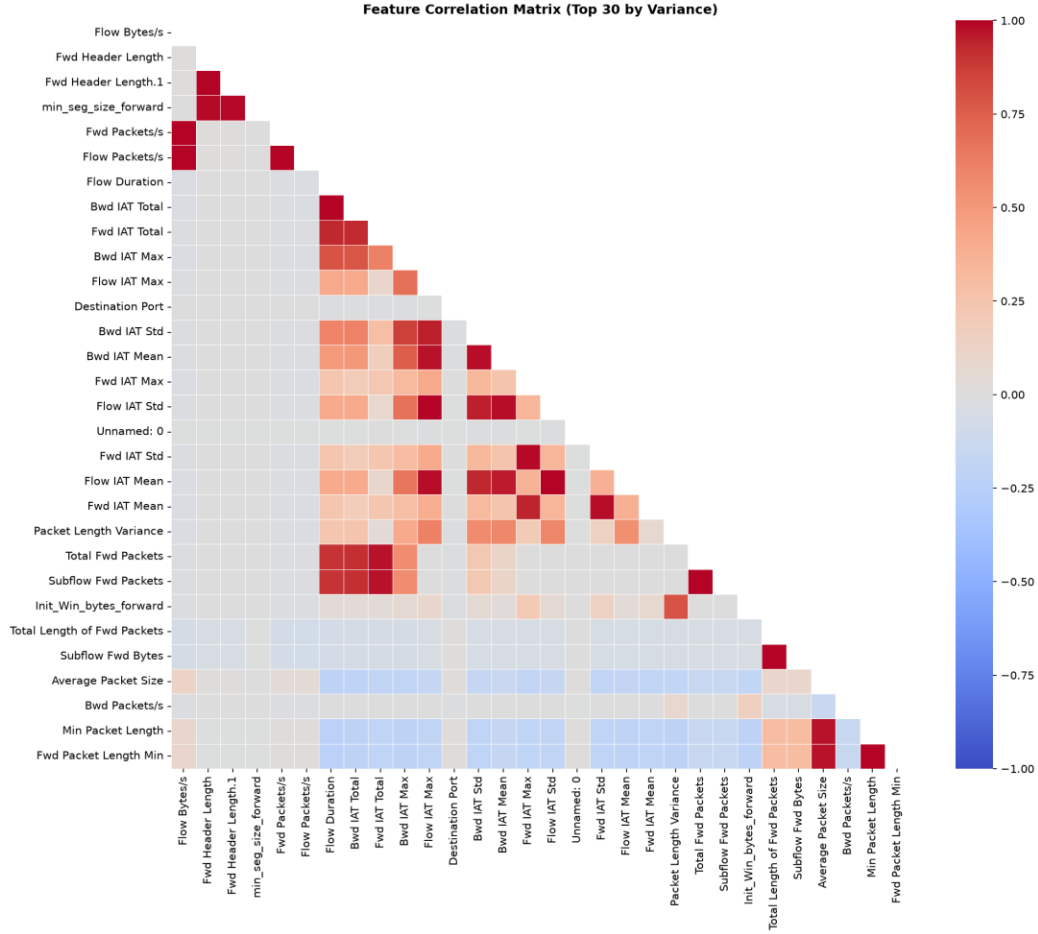


Figure 2. Pairwise correlation matrix of the thirty highest-variance flow features, computed on training-session data only. Strongly correlated clusters (e.g., IAT and Fwd/Bwd packet-count groups) motivate the semantic feature grouping used in FlowFusion's structural and feature-interaction branches (Section 4.3).

4.3 FlowFusion: Proposed Multi-View Architecture

The proposed **FlowFusion** architecture processes the same 70-dimensional flow feature vector through three distinct branches, each designed to learn complementary feature representations. The resulting outputs are combined using cross-branch attention and a scalar gating mechanism to produce the final representation. The term **multi-view** is used throughout this report because all branches process the same input feature vector. Each branch learns a different representation of the shared data instead of using separate data modalities.

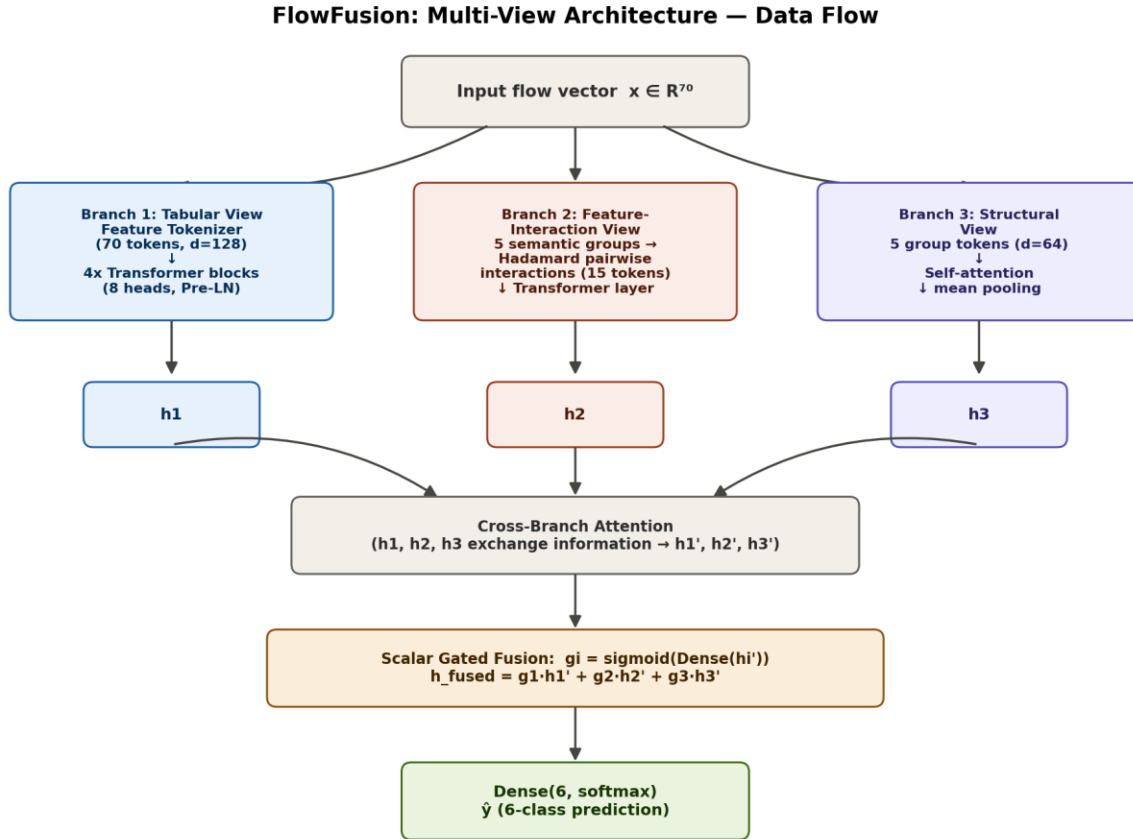


Figure 3. FlowFusion architecture and data-flow diagram. The 70-dimensional input is routed unchanged into three branches (tabular, feature-interaction, structural); branch outputs h_1 , h_2 , h_3 exchange information via cross-branch attention, are combined through interpretable scalar gates (g_1 , g_2 , g_3), and the fused representation is passed to the final softmax classification layer.

Branch 1 — Tabular View (Feature-Tokenizer Self-Attention)

In this branch, we transformed each of 70 input features into a 128 dimensional embedding, forming a sequence of feature tokens. Then we processed the four tranformer encoder layers with eight attention heads and a Pre-LayerNorm architecture. This allows the model to find dependencies between features, for example, the relationship between packet sizes and flag counts. The final [CLS] token is used as the branch output.

Branch 2 — Feature-Interaction View

In this branch, the input features are organised into five semantic groups , Volume, Rate, Flags, Inter-Arrival Time (IAT), and Window. We computed Pairwise Hadamard interactions between every group pair, together with the original group representations, which results in 15 interaction tokens. These tokens are processed by a Transformer kayer to capture relationships between different feature groups and identify the interactions that contribute most to the classification task. The grouping of related features in this manner provides a structured way to model cross group dependencies instead of treating all features as a single set.

Branch 3 — Structural View (Feature-Group Attention)

In this branch, the input features are first divided into the same five semantic groups. Each group is converted into a 64 dimensional feature representation, after which a multi head self attention layer learns the relationships among these groups. Finally, we applied a mean pooling to combine the learned information into a single feature vector. Unlike the first branch, which focuses on individual features, this branch captures interactions at the group level.

Cross-Branch Attention and Gated Fusion

Before combining the outputs, each branch exchanges information with the other two through a cross-branch attention mechanism. The updated feature representations are then passed to a gating module, which assigns a single importance score to each branch using a sigmoid activation. These weighted representations are combined to form a unified feature vector, which is finally passed to a Dense layer with a softmax activation for six-class classification. Using a single gate value for each branch also makes the model easier to interpret, as the gate scores indicate the relative contribution of each branch to the final prediction.

We trained FlowFusion using a **class-weighted focal loss**, which gives greater importance to difficult training samples while reducing the influence of samples that are already classified correctly. This helps the model learn better from challenging classes, particularly **DrDoS_MSSQL**, which is harder to classify. The model is optimized using the **AdamW** optimizer with a learning rate of 1.5×10^{-4} . To improve training stability and prevent overfitting, a linear warmup of 1,200 steps, **ReduceLROnPlateau** learning rate scheduling, gradient clipping with a maximum norm of 1.0, and early stopping with a patience of 10 epochs are employed. The overall training process is summarized in **Algorithm 1**.

```

Algorithm 1: FlowFusion Forward Pass
Input: flow feature vector  $x \in \mathbb{R}^{70}$ 
1:  $t1 \leftarrow \text{FeatureTokenizer}(x)$  // 70 tokens,  $d=128$ 
2:  $h1 \leftarrow \text{TransformerEncoder}(t1)[\text{CLS}]$  // tabular view
3:  $t2 \leftarrow \text{InteractionTokens}(\text{GroupSplit}(x))$  // 15 tokens (5 groups +  $C(5,2)$  pairs)
4:  $h2 \leftarrow \text{TransformerEncoder}(t2)$  // feature-interaction view
5:  $t3 \leftarrow \text{GroupEmbed}(\text{GroupSplit}(x))$  // 5 tokens,  $d=64$ 
6:  $h3 \leftarrow \text{SelfAttention}(t3) \rightarrow \text{MeanPool}$  // structural view
7:  $h1', h2', h3' \leftarrow \text{CrossBranchAttention}(h1, h2, h3)$ 
8: for  $i$  in  $\{1,2,3\}$ :  $g_i \leftarrow \text{sigmoid}(\text{Dense}_1(h_i'))$  // interpretable scalar gates
9:  $h_{\text{fused}} \leftarrow g_1 \cdot h1' + g_2 \cdot h2' + g_3 \cdot h3'$ 
10:  $\hat{y} \leftarrow \text{Softmax}(\text{Dense}_6(h_{\text{fused}}))$ 
return  $\hat{y}, (g1, g2, g3)$ 

```

Algorithm 1. Forward pass of the proposed FlowFusion architecture.

4.4 Baseline Models

To provide a fair comparison, four baseline models are trained using the same dataset, preprocessing steps, and experimental settings as the proposed FlowFusion framework. We configured the XGBoost with a maximum tree depth of 10, while LightGBM uses up to 255 leaves per tree. Both models use a learning rate of 0.1, row and columns sampling of 0.8. It used L1/L2 regularization and early stopping to improve generalization. In addition, LightGBM uses class balanced weights as a safeguard against any potential discrepancies during stratified sampling. FlowMLP is a residual multi-layer perceptron with four hidden

layers containing 512, 256, 128, and 64 neurons. It uses layer normalization, Swish activation, dropout, Mixup, and Gaussian noise augmentation during training. The model is optimized with the Adam optimizer and label smoothing.

FT-Transformer is a standalone Feature-TOKENizer Transformer that follows the same tokenization and self-attention approach as the tabular branch of FlowFusion (Section 4.3). It is trained using the AdamW optimizer along with label smoothing and a linear warmup schedule.

4.5 Ensemble: FlowFusion + LightGBM

Among the individual models, LightGBM gave the highest Macro-F1 score. FlowFusion, on the other hand, processes different groups of network traffic features separately before combining them. To improve the overall prediction performance, both models were combined using a soft-voting ensemble. The final prediction is calculated as:

$$p_{final} = \alpha p_{FlowFusion} + (1 - \alpha) p_{LightGBM}$$

The value of α was selected using grid search on the validation set. The best performance was obtained with $\alpha = 0.41$, which gives **41% weight to FlowFusion** and **59% weight to LightGBM**. The performance of the final ensemble is presented in Section 5.6, along with the per-class error analysis.

4.6 Explainability

Two explainability techniques are used in this study. TreeSHAP is applied to the gradient-boosted models to measure how much each feature contributes to the final prediction. TreeSHAP uses the structure of the trained trees to compute exact Shapley values efficiently, without the need for sampling. For the neural models (FlowMLP, FT-Transformer, and FlowFusion), Integrated Gradients is used. Integrated Gradients works by interpolating between a zero-valued baseline input and the actual input, and accumulating the model's gradients along this path to assign importance scores to each feature. It estimates the importance of each input feature by accumulating gradients from an all-zero baseline to the actual input. In addition, FlowFusion provides the values of its three gating weights for every prediction. These values indicate how much each feature branch contributes to the final output and provide another way to understand the model's decisions.

5. Experimental Details

5.1 Setup and Evaluation Protocol

We trained all models on a NVIDIA T4 GPU using Lightning AI Studio. Five different random seeds (42, 123, 456, 789, and 1024) were used to account for variations in weight initialization, data shuffling, and stochastic regularization. The final results are reported as the mean $\pm$ standard deviation across these five runs. The Wilcoxon signed-rank test was used to compare the models using the same set of random seeds.

We chose Macro-F1 as the primary evaluation metric because it gives equal importance to every class. Since the March-11 test set is class-imbalanced, accuracy alone may not represent performance across all

classes. Along with Macro-F1, accuracy, weighted-F1, macro precision, macro recall, and one-vs-rest ROC-AUC are also reported.

Hyperparameters for all models were tuned using a 30-trial Optuna search with the TPE sampler and median pruning on the validation set. The best hyperparameters obtained from this search were used for the final multi-seed experiments.

5.2 Why a Cross-Session Split

A key design choice in this work was to train and test the models on different capture sessions. The models were trained using only the January 12 traffic and tested only on the March 11 traffic instead of using a random split from a combined dataset.

Network traffic changes across different capture sessions because the active users and running applications are not always the same. If both the training and testing data come from the same session, the model may learn session-specific patterns that do not generalize well to new traffic.

This evaluation setup usually produces lower performance than a random same-session split. However, it provides a more realistic estimate of how the model is expected to perform in real deployment.

5.3 Main Results

Table 1 reports the primary comparison of all four baselines and the proposed FlowFusion model on the held-out March-11 session (mean $\pm$ std across 5 seeds).

Model	Accuracy	Macro-F1	Weighted-F1	ROC-AUC
LightGBM	0.9880 $\pm$ 0.0001	0.9891 $\pm$ 0.0001	0.9880 $\pm$ 0.0001	0.9990 $\pm$ 0.0000
XGBoost	0.9876 $\pm$ 0.0002	0.9887 $\pm$ 0.0002	0.9876 $\pm$ 0.0002	0.9990 $\pm$ 0.0000
FT-Transformer	0.9874 $\pm$ 0.0004	0.9884 $\pm$ 0.0005	0.9874 $\pm$ 0.0004	0.9987 $\pm$ 0.0001
FlowFusion (proposed)	0.9872 $\pm$ 0.0005	0.9883 $\pm$ 0.0005	0.9872 $\pm$ 0.0005	0.9987 $\pm$ 0.0001
FlowMLP	0.9860 $\pm$ 0.0007	0.9871 $\pm$ 0.0007	0.9860 $\pm$ 0.0007	0.9984 $\pm$ 0.0003

Table 1. Model comparison on the held-out March-11 test session (5-seed mean $\pm$ std).

The spread between the strongest model (LightGBM, 0.9891) and the weakest (FlowMLP, 0.9871) is only 0.0020 — every architecture, from gradient-boosted trees to multi-view FlowFusion, achieves closely clustered performance under the cross-session protocol. Narrow per-seed standard deviations (0.0001–0.0007) confirm this ranking is stable, not seed noise. Both tree ensembles rank above every deep model; FlowFusion is competitive with FT-Transformer and slightly ahead of FlowMLP, suggesting that the three-branch design adds some value over a single encoder.

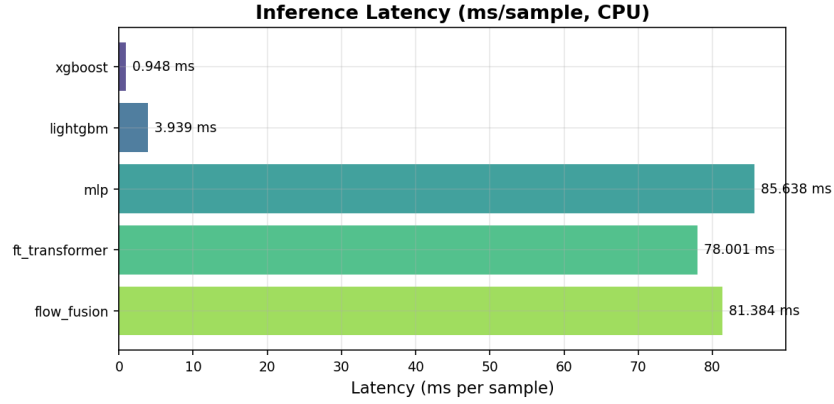


Figure 4. Per-sample CPU inference latency across all five models. XGBoost (0.95 ms) and LightGBM (3.94 ms) are one to two orders of magnitude faster than every deep model (78–86 ms), discussed further in Section 5.8.

This latency gap matters for deployment and is revisited in Section 5.8: beyond matching or exceeding the deep models on Macro-F1, XGBoost and LightGBM classify each flow roughly 20–85× faster on CPU — directly relevant to line-rate detection.

5.4 Branch Ablation Study

To validate that all three branches of FlowFusion contribute to the fused representation, eight configurations were evaluated: each branch alone, each pairwise combination, and the full model under concatenation versus scalar gated fusion (Table 2).

Configuration	Branches	Fusion	Macro-F1	Accuracy
Tabular only	B1	gated	0.9877	0.9867
Feature-Interaction only	B2	gated	0.9734	0.9746
Structural only	B3	gated	0.9839	0.9833
B1 + B2	two-branch	gated	0.9875	0.9863
B1 + B3	two-branch	gated	0.9876	0.9867
B2 + B3	two-branch	gated	0.9823	0.9824
Full model	B1+B2+B3	concatenation	0.9880	0.9868
Full model (proposed)	B1+B2+B3	scalar gated	0.9881	0.9868

Table 2. Branch ablation study validating the FlowFusion architecture.

The tabular branch (B1) is clearly the strongest single view (0.9877), the structural branch (B3) is second (0.9839), and the feature-interaction branch (B2) is weakest alone (0.9734). Yet the full three-branch, gated-fusion model outperforms every single-branch and pairwise configuration (0.9881), confirming B2 still contributes complementary signal once combined under gating — precisely why the three-branch design is retained as the proposed architecture.

5.5 Per-Class Results

Class	F1 (Mean)	F1 (Std)
BENIGN	0.9945	0.0012
DrDoS_LDAP	0.9831	0.0009
DrDoS_MSSQL	0.9705	0.0006
DrDoS_NetBIOS	0.9989	0.0001

Class	F1 (Mean)	F1 (Std)
DrDoS_UDP	0.9863	0.0006
Syn	0.9988	0.0002

Table 3. FlowFusion per-class F1 on the test session (5-seed mean $\pm$ std).

DrDoS_MSSQL is the hardest class (F1 = 0.9705) against F1 > 0.98 for every other class; this is the class the focal-loss objective (Section 4.3) specifically targets, and it remains the primary class the ensemble (Section 5.6) is able to further improve upon.

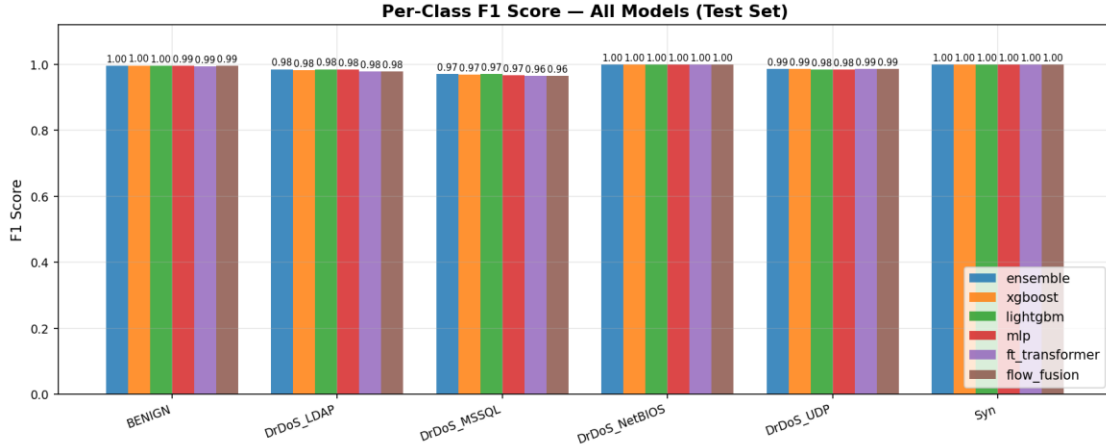


Figure 5. Per-class F1 score across all six models (ensemble, XGBoost, LightGBM, FlowMLP, FT-Transformer, FlowFusion) on the held-out test session, confirming DrDoS_MSSQL as the consistently hardest class across every model and the ensemble's advantage being most visible on this class.

5.6 Ensemble Results

Table 4 compares FlowFusion alone, LightGBM alone, and the soft-voting ensemble ($\alpha = 0.41 : 0.59$).

Metric	FlowFusion Alone	LightGBM Alone	Ensemble
Macro-F1	0.9887	0.9890	0.9895
Accuracy	0.9877	0.9878	0.9885
BENIGN F1	0.9949	0.9962	0.9955
DrDoS_MSSQL F1	0.9700	0.9698	0.9715
DrDoS_NetBIOS F1	0.9989	0.9993	0.9994

Table 4. Ensemble performance versus its two constituent models.

The ensemble performs better than both individual models in terms of Macro-F1 and accuracy. It also achieves higher F1 scores for the DrDoS_MSSQL and DrDoS_NetBIOS classes, while LightGBM performs slightly better only for the BENIGN class.

This improvement is likely because the two models make different types of prediction errors. LightGBM separates classes such as DrDoS_LDAP and DrDoS_MSSQL effectively using its tree-based decision boundaries. In contrast, FlowFusion combines information from all three feature branches, which helps improve the prediction of BENIGN, DrDoS_UDP, and Syn traffic. Since both models capture different characteristics of the data, combining their predictions through soft voting improves the overall performance. As a result, the ensemble achieves the best performance among all the models evaluated in this study.

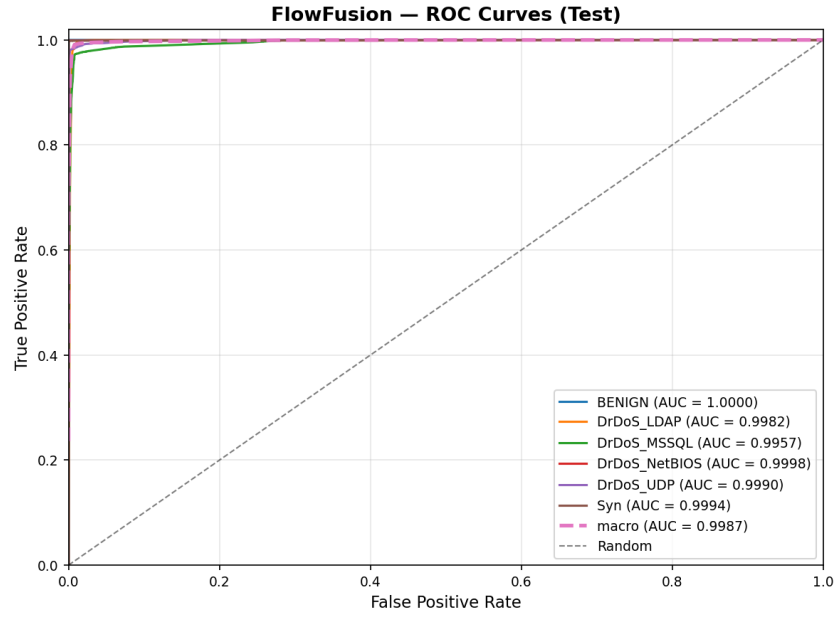


Figure 6. One-vs-rest ROC curves for FlowFusion on the six-class test session (macro-averaged AUC = 0.9987).

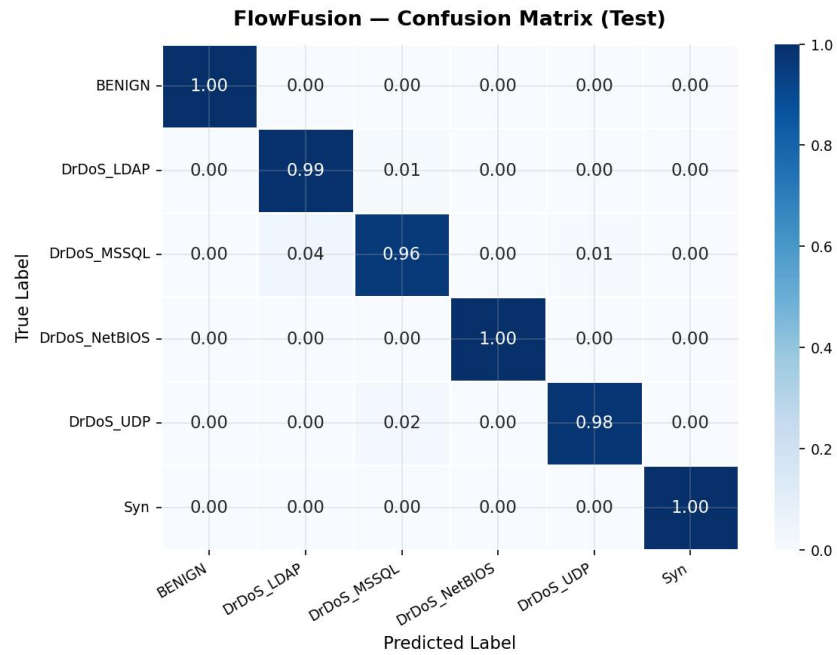


Figure 7. Confusion matrix for FlowFusion on the held-out test session, showing residual misclassification concentrated around the DrDoS_MSSQL class (4% predicted as DrDoS_LDAP, 2% as DrDoS_UDP).

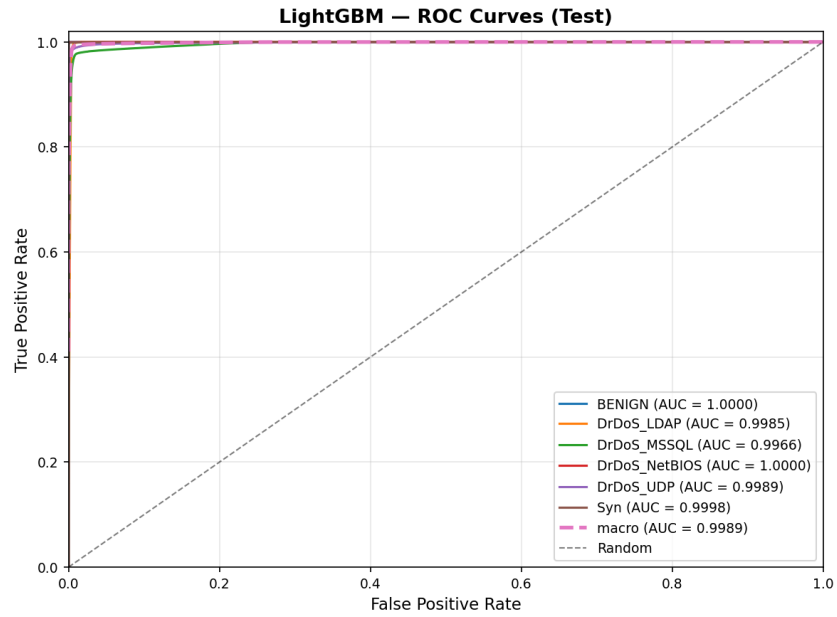


Figure 8. One-vs-rest ROC curves for LightGBM, the strongest standalone model in this study (macro-averaged AUC = 0.9989).

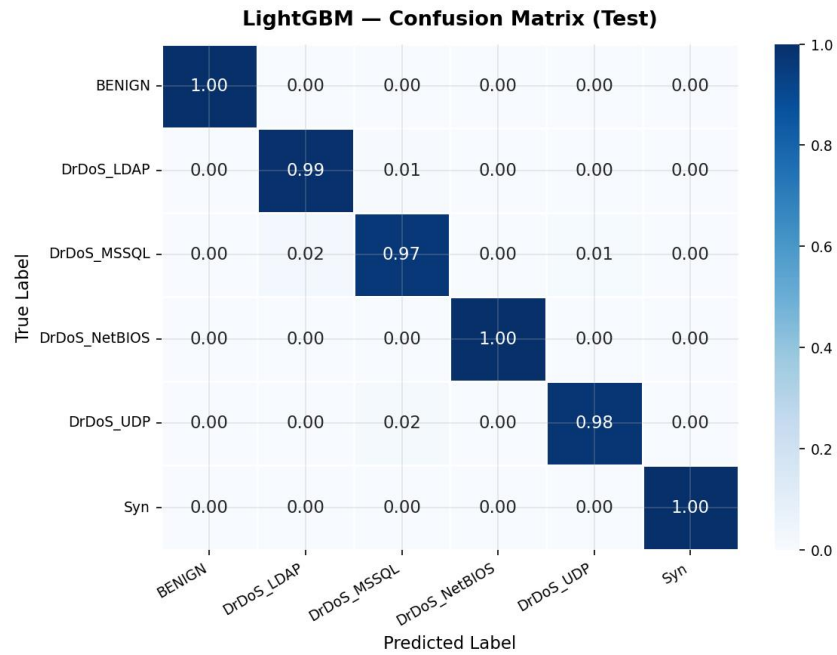


Figure 9. Confusion matrix for LightGBM on the held-out test session.

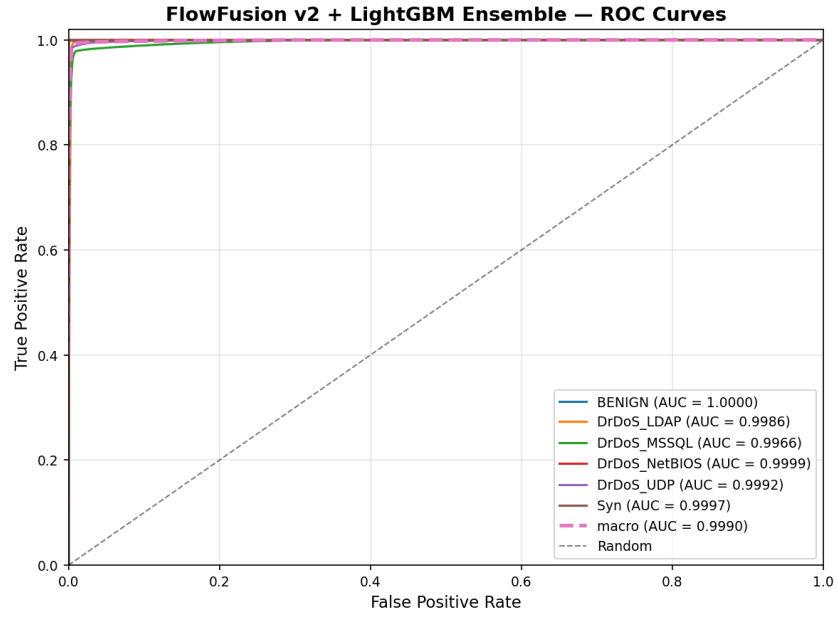


Figure 10. One-vs-rest ROC curves for the FlowFusion + LightGBM soft-voting ensemble ($\alpha = 0.41:0.59$), the best-performing configuration in this study (macro-averaged AUC = 0.9990).

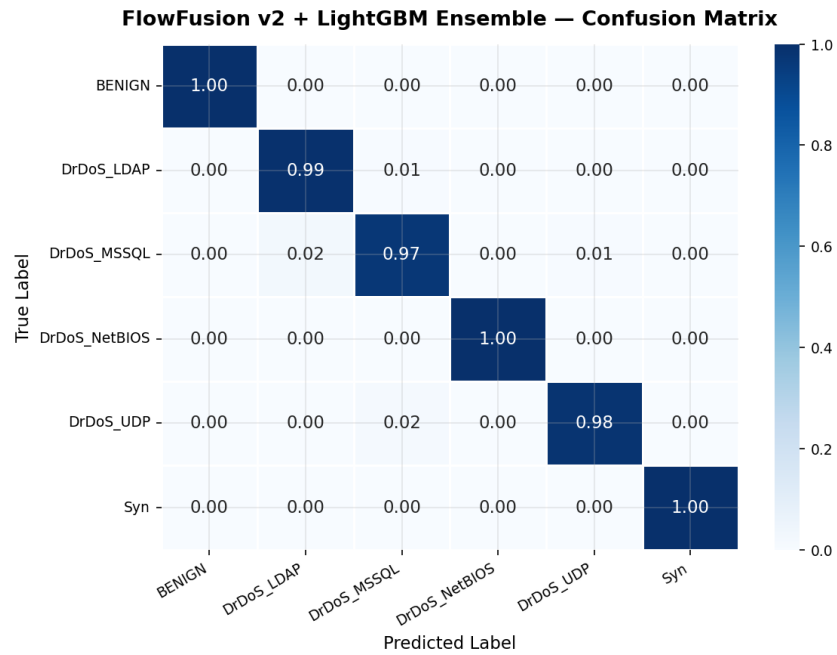


Figure 11. Confusion matrix for the FlowFusion + LightGBM ensemble. Relative to Figures 7 and 9, DrDoS_MSSQL confusion is reduced to 2% (versus 4% for FlowFusion and a comparable rate for LightGBM), the clearest class-level illustration of the ensemble's complementary gain.

5.7 Interpretability Analysis

FlowFusion generates scalar gate values for every prediction. These values are averaged over the test set to analyze the contribution of each branch (Figure 12).

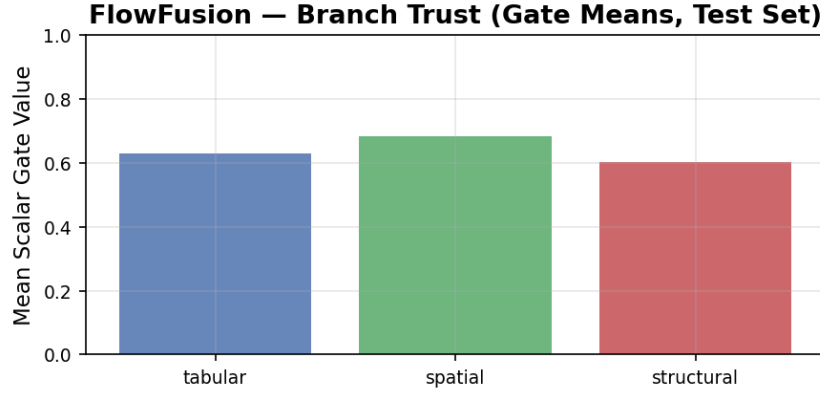


Figure 12. Mean learned gate values for FlowFusion's three branches, aggregated over the test set. The branch labeled "spatial" in the plot corresponds to the feature-interaction branch described in Section 4.3.

Note: The branch labeled "spatial" in the figure corresponds to the feature-interaction branch described in Section 4.3.

The average gate values are **0.63** for the tabular branch, **0.68** for the feature-interaction branch, and **0.60** for the structural branch. These values are fairly balanced, indicating that the model uses information from all three branches instead of relying mainly on a single branch.

An interesting observation is that the feature-interaction branch has the lowest standalone Macro-F1 score (Table 2, **0.9734**), but it receives the highest average gate value after fusion. This suggests that the branch provides information that complements the other two branches, even though its individual performance is lower.

This observation is also supported by the ablation study (Section 5.4), where the complete FlowFusion model performs better than all reduced versions. Overall, these results suggest that the gating mechanism combines information from all three branches instead of depending mainly on the strongest standalone branch.

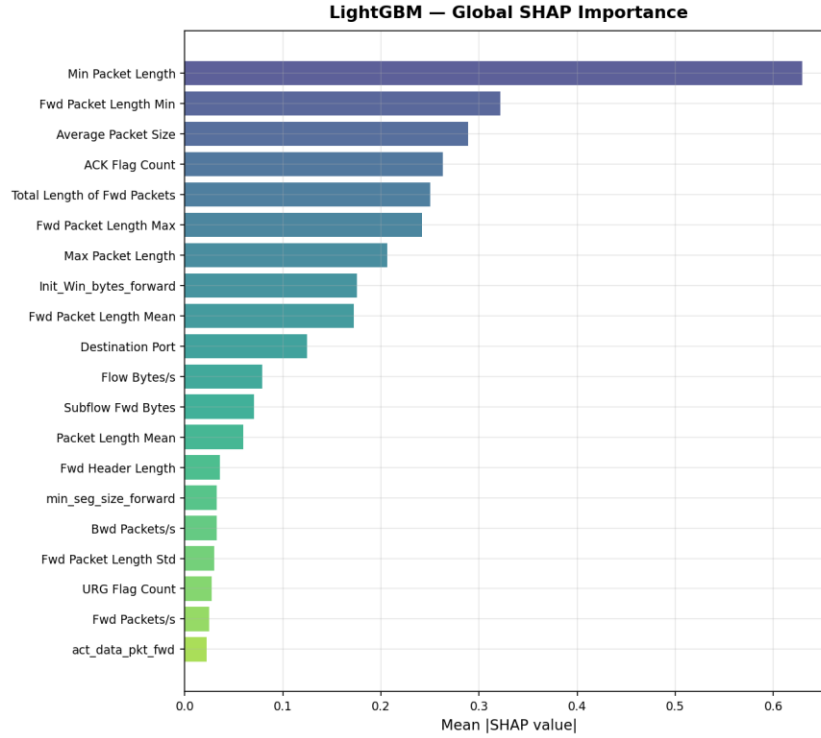


Figure 13. Global TreeSHAP feature importance for LightGBM, ranking flow-level features by mean absolute SHAP value across the test session. Min Packet Length, Fwd Packet Length Min, and Average Packet Size dominate, followed by ACK Flag Count and total forward packet-length statistics.

The TreeSHAP ranking in Figure 13 supports the feature grouping used in FlowFusion. Four of the six most important features (Min Packet Length, Fwd Packet Length Min, Average Packet Size, and Total Length of Fwd Packets) belong to the **Volume** feature group used in the structural and feature-interaction branches (Section 4.3). In addition, **ACK Flag Count** belongs to the **Flags** feature group. This suggests that the manually defined feature groups capture useful information and are meaningful for the prediction task rather than being arbitrary feature divisions.

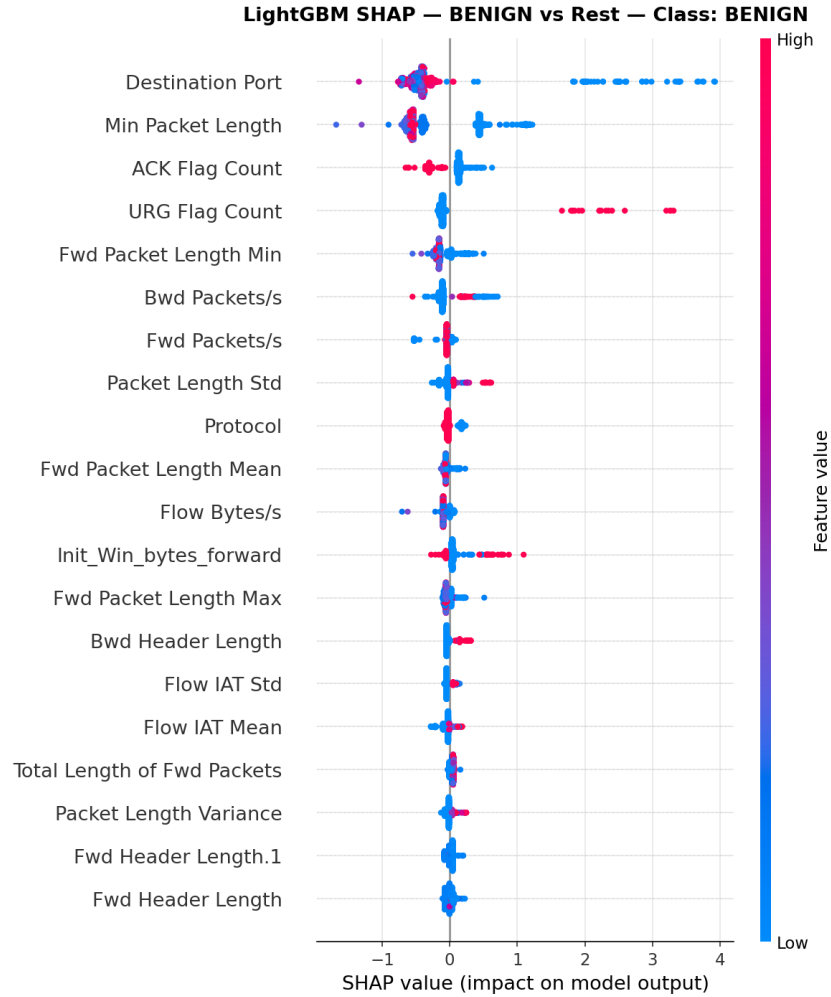


Figure 14. TreeSHAP summary plot for the **BENIGN** class under LightGBM. Each point is one test-session flow; horizontal position shows the SHAP value (impact on the **BENIGN** prediction) and colour shows the feature's value for that flow.

Figure 14 shows that lower values of **Destination Port** and **Min Packet Length** increase the model's prediction toward the **BENIGN** class, while higher values of these features reduce it. This matches the traffic pattern in the dataset, where attack traffic is concentrated on specific service ports and often uses minimal or padded packet sizes for amplification. Higher **URG Flag Count** values also increase the prediction toward the **BENIGN** class. This suggests that the URG flag appears less frequently or less consistently in the DDoS traffic included in this dataset.

5.8 Discussion: Why Gradient-Boosted Trees Remain Competitive

Table 1 shows that both tree-based ensemble models perform better than all the deep learning models, including FlowFusion. This result may be explained by two main factors. First, the input data consists only of structured per-flow tabular features, which are well suited to tree-based models. Trees make decisions directly using feature values, whereas neural networks first learn feature representations. In addition, gradient boosting generally handles skewed and long-tailed feature distributions well, which is useful for

the 70 features used in this dataset. Deep learning models, including FlowFusion, first learn feature representations from these tabular statistics and can be more sensitive to feature distributions containing many outliers, which is one of the reasons for using RobustScaler (Section 4.2). Another factor is the cross-session split, which introduces temporal distribution shift. Tree-based ensemble models generally handle this type of shift better than deep learning models. This can be seen in the BENIGN class, where all deep learning models perform worse than the tree-based models. This may be because BENIGN traffic changes more across different capture sessions (Table 3).

Although FlowFusion does not outperform LightGBM as a standalone model, it provides two important advantages in this study. First, it offers per-prediction interpretability through its gate values (Figure 12), making it easier to understand how the different feature branches contribute to each prediction. Second, when combined with LightGBM in a soft-voting ensemble, it improves the overall performance and achieves the best result in this study (Table 4). These findings suggest that the main strength of FlowFusion for this dataset lies in its interpretability and its complementary contribution to the ensemble rather than its standalone performance.

6. Conclusion

This report investigated DDoS detection on the CIC-DDoS2019 dataset using a cross-session evaluation protocol. The models were trained on the January 12 capture and evaluated only on the March 11 capture to avoid data leakage between training and testing. Five models were evaluated under the same experimental setup using five random seeds: XGBoost, LightGBM, FlowMLP, FT-Transformer, and the proposed FlowFusion model. FlowFusion combines three feature views—a tabular self-attention branch, a feature-interaction branch, and a structural feature-group branch—using cross-branch attention and scalar gating. The model was trained using a focal loss objective. LightGBM achieved the highest standalone Macro-F1 score (**0.9891**) while requiring much lower inference time than the deep learning models. It performed slightly better than XGBoost, FT-Transformer, and FlowFusion (**0.9883**). When FlowFusion and LightGBM were combined using a soft-voting ensemble with weights of **0.41** and **0.59**, the ensemble achieved the best overall Macro-F1 score (**0.9895**). This suggests that the two models learn different patterns from the data, allowing the ensemble to improve the overall performance. The branch ablation study and gate-value analysis suggest that all three branches contribute to the final model. The average gate values (0.63, 0.68, and 0.60) are fairly balanced, indicating that the model does not rely mainly on a single branch. In addition, the gate values make it possible to understand how each branch contributes to individual predictions, providing a level of interpretability that is not available from the tree-based ensemble models. One important finding of this study is that the gradient-boosted tree models slightly outperform the more complex FlowFusion architecture on this dataset. This shows that a more complex deep learning model does not always perform better than a well-tuned tree-based model. For this dataset, FlowFusion is mainly useful because it provides interpretable gate values that help explain the contribution of each branch to the final prediction. It also improves the overall performance when combined with LightGBM in a soft-voting ensemble.

There are a few directions we think are worth exploring next. First, for real deployment, the multi-view architecture would need to be optimized for speed possibly through model compression or quantization so it can match the sub-millisecond latency of LightGBM while keeping the interpretability from the gate values. Second, DDoS attacks are distributed by nature, and modeling traffic as a graph (where flows are nodes and relationships are edges) using Graph Neural Networks could capture coordination patterns between flows that per-flow features miss entirely. Third, since we observed temporal drift between the January and March captures, continual learning methods could help the model adapt to changing traffic patterns without forgetting previously learned attack signatures. Finally, adversarial robustness needs to be evaluated attackers could modify features like inter-arrival times or packet padding to bypass flow-based detectors.

References

- [1] D. Kumar, R. K. Pateriya, R. K. Gupta, V. Dehalwar, and A. Sharma, “DDoS Detection using Deep Learning,” *Procedia Computer Science*, vol. 218, pp. 2420–2429, 2023, International Conference on Machine Learning and Data Engineering.
- [2] S. Aktar and A. Y. Nur, “Towards DDoS Attack Detection using Deep Learning Approach,” *Computers & Security*, vol. 129, p. 103251, 2023.
- [3] Meenakshi, K. Kumar, and S. Behal, “Distributed Denial of Service Attack Detection using Deep Learning Approaches,” in *Proc. 8th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, New Delhi, India, 2021, pp. 491–495.
- [4] Canadian Institute for Cybersecurity, “DDoS Evaluation Dataset (CIC-DDoS2019),” University of New Brunswick, 2019. [Online]. Available: <https://www.unb.ca/cic/datasets/ddos-2019.html>